Lecture 01: Introduction to Algorithms and Data Structures

TKU211231 — Student Lecture Notes

Data Structures and Algorithms Course (TKU211231)

February 12, 2026

Contents

1 What is an Algorithm? 3
- 1.1 Formal Definition . . . 3
- 1.2 Algorithm Examples . . . 4

2 What are Data Structures? 5
- 2.1 Formal Definition . . . 6
- 2.2 Common Data Structures Overview . . . 6

3 Why Study Algorithms and Data Structures? 8
- 3.1 Practical Importance . . . 8

4 Algorithm Analysis: The Basics 9
- 4.1 Why Analyze Algorithms? . . . 9
- 4.2 Measuring Efficiency . . . 10
- 4.3 The Word-RAM Computation Model . . . 10
- 4.4 Asymptotic Notation . . . 11
- 4.5 Common Growth Rates . . . 12
- 4.6 Big-Omega and Big-Theta Notation . . . 13
- 4.7 Best, Average, and Worst Case . . . 14

5 Problem-Solving Strategies 15
- 5.1 Common Strategies . . . 15

6 Summary Tables and Quick Reference 18
- 6.1 Algorithm Properties Checklist . . . 18
- 6.2 Common Time Complexities . . . 18
- 6.3 Data Structure Operation Costs . . . 18

7 Practice Problems 19
- 7.1 Basic Concepts . . . 19
- 7.2 Algorithm Analysis . . . 19
- 7.3 Data Structures . . . 20
- 7.4 Problem-Solving Strategies . . . 21
- 7.5 Challenge Problem . . . 21

8 Closing Thoughts **22**

Opening Story: The Recipe That Changed Computing

Story

In the 9th century, a Persian mathematician named Muhammad ibn Musa al-Khwarizmi wrote a groundbreaking book on solving equations. The Latin translation of his name, "Algorithmi," eventually gave us the word "algorithm." But why does this matter today?

Imagine you're planning a road trip across Indonesia. You have dozens of cities to visit, limited time, and you want to minimize fuel costs. Should you visit Jakarta first or Surabaya? What's the optimal route? This isn't just about following directions—it's about finding the *best* solution among millions of possibilities.

This is what algorithms do: they provide systematic, step-by-step procedures to solve problems efficiently. And data structures? They're the organizational tools that make these algorithms possible. Together, they form the foundation of every app, website, and system you interact with daily.

Key Point

Think about your morning routine. The sequence of steps you follow to get ready is essentially an algorithm! Algorithms are everywhere in daily life, not just in computers.

1 What is an Algorithm?

Story

Before we define an algorithm formally, let me tell you about Grace Hopper, one of the pioneers of computer programming. She often said, "The most dangerous phrase in the language is, 'We've always done it this way.'"

Why? Because algorithms are all about finding better ways to solve problems. The algorithm your grandmother used to sort her recipe cards might work, but is it the *best* way when you have 10,000 recipes? Understanding algorithms means understanding not just *how* to solve a problem, but *how well* we're solving it.

1.1 Formal Definition

Definition 1.1 (Algorithm)**.** *An **algorithm** is a finite sequence of well-defined, unambiguous instructions that, when executed, solves a particular problem or performs a specific computation. An algorithm must satisfy the following properties:*

1. ***Finiteness**: The algorithm must terminate after a finite number of steps*
2. ***Definiteness**: Each step must be precisely defined with no ambiguity*
3. ***Input**: The algorithm has zero or more inputs*

4. ***Output***: *The algorithm produces at least one output*
5. ***Effectiveness***: *All operations must be sufficiently basic that they can be done exactly and in finite time*

Key Point

An algorithm is like a recipe, but more precise. Every step must be clear enough that a computer (or person) can execute it without any confusion or guessing.

Important

All five properties are *required*. If even one is missing, you don't have a valid algorithm. For example, "Add salt to taste" fails the definiteness requirement because "to taste" is subjective and ambiguous.

Exercise

Exercise 1.1: Determine whether each of the following constitutes a valid algorithm. For each, identify which properties (if any) are violated:

(a) A recipe that says "Add salt to taste"

(b) GPS directions: "Head north on Main Street, turn right at the traffic light"

(c) An instruction: "Keep dividing the number by 2 until it equals zero"

(d) The process of withdrawing money from an ATM

(e) An instruction: "Think of a number, any number"

Hint: Check each against the five properties: finiteness, definiteness, input, output, and effectiveness.

1.2 Algorithm Examples

Example 1.1 (Finding Maximum in a List)**.** *Let's develop an algorithm to find the maximum value in a list of numbers.*

Problem: *Given a list A of n numbers, find the largest value.*

Algorithm: FindMax

1. *Let* $max \leftarrow A[1]$ *(assume first element is maximum)*
2. *For* $i \leftarrow 2$ *to* n*:*
   (a) *If* $A[i] > max$*, then* $max \leftarrow A[i]$
3. *Return max*

Verification:

- Finiteness*: Loops exactly* $n - 1$ *times, then terminates*

- Definiteness*: Each comparison and assignment is clearly defined*
- Input*: List A of n numbers*
- Output*: Single number (maximum value)*
- Effectiveness*: All operations (comparison, assignment) are basic*

Note

Let's trace through FindMax with a concrete example: $A = [3, 7, 2, 9, 1, 5]$

Iteration	i	A[i]	max	Comparison
Initial	—	—	3	(first element)
1	2	7	7	$7 > 3$? YES
2	3	2	7	$2 > 7$? NO
3	4	9	9	$9 > 7$? YES
4	5	1	9	$1 > 9$? NO
5	6	5	9	$5 > 9$? NO

Final result: max $= 9$

Exercise

Exercise 1.2: Trace through the FindMax algorithm with the following input arrays:

(a) $A = [5, 2, 8, 1, 9]$

(b) $A = [10, 9, 8, 7, 6]$ (already sorted descending)

(c) $A = [1, 2, 3, 4, 5]$ (already sorted ascending)

For each, create a table showing the value of `max` after each iteration.

2 What are Data Structures?

Story

Imagine walking into a massive library where all the books are piled randomly on the floor. You need to find a book on "Indonesian History." How long would it take? Hours? Days?
Now imagine the same library with books organized on shelves, sorted by subject, then by author. Finding that same book might take minutes.
This is exactly what data structures do for computer programs. They organize information so that algorithms can access, modify, and process it efficiently. The right data structure can turn an impossibly slow program into one that runs instantly.

2.1 Formal Definition

Definition 2.1 (Data Structure)**.** *A **data structure** is a specialized format for organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. A data structure provides:*

1. ***Organization**: A systematic way to store related data elements*
2. ***Access methods**: Defined operations to retrieve and manipulate data*
3. ***Relationships**: Rules governing how data elements relate to each other*
4. ***Constraints**: Limitations on what operations are allowed*

Key Point

Data structures are the "containers" that hold our data. Just as you wouldn't store soup in a paper bag, you need to choose the right data structure for your problem. The wrong choice can make even simple operations painfully slow.

2.2 Common Data Structures Overview

Data structures can be broadly classified into two categories:

- **Linear Data Structures**: Elements arranged sequentially
  - Array: Fixed-size, indexed collection
  - List: Ordered, resizable sequence
  - Stack: LIFO (Last In, First Out)
  - Queue: FIFO (First In, First Out)
- **Non-Linear Data Structures**: Elements arranged hierarchically or as networks
  - Tree: Hierarchical structure
  - Graph: Network of connections

Example 2.1 (Array Data Structure)**.** *An **array** is a contiguous block of memory that stores elements of the same type, accessed by numeric indices.*

***Visual Representation**:*

```
Index:     0    1    2    3    4
Array A:  [10] [25] [37] [42] [58]
```

***Basic Operations**:*

- ***Access**: $A[i]$ retrieves element at index i (constant time: $O(1)$)*
- ***Update**: $A[i] \leftarrow x$ sets element at index i to value x (constant time: $O(1)$)*
- ***Search**: Finding value x requires checking each element (linear time: $O(n)$)*

***Key Properties**:*

- *Fixed size (in most languages)*
- *Fast random access*
- *Elements stored contiguously in memory*
- *Indexing starts at 0 in most languages*

Important

Arrays provide $O(1)$ access time because the memory address of any element can be calculated directly:

$$\text{address}(A[i]) = \text{base_address} + (i \times \text{element_size})$$

This mathematical property makes arrays incredibly efficient for random access.

Exercise

Exercise 1.3: For each scenario below, suggest an appropriate data structure and explain why:

(a) You're building a browser's "Back" button functionality

(b) You need to store a phone directory that requires fast lookups by name

(c) You're managing a printer queue where jobs must be processed in order received

(d) You want to represent the organizational hierarchy of a company (CEO, managers, employees)

(e) You need to store a social network where people are connected as friends

Hint: Think about what operations are most common (insertion, deletion, search) and what ordering constraints exist (LIFO, FIFO, hierarchical).

3 Why Study Algorithms and Data Structures?

Story

In 2006, Google engineer Jon Bentley posed a challenge to Donald Knuth, one of computer science's legends: "How would you print the k most common words in a file?"
Knuth's solution was elegant, but more importantly, it demonstrated deep understanding of both algorithms and data structures. The difference between a naive solution and an optimized one? The naive approach might take hours on a large file; Knuth's approach took seconds.
This isn't academic exercise—this is the difference between a system that scales and one that collapses under load. When Instagram needs to display your feed, when Netflix recommends movies, when Google Maps finds your route—all of these rely on carefully chosen algorithms and data structures.

3.1 Practical Importance

Key Point

The choice of algorithm and data structure matters more than most programmers realize. A good algorithm can be thousands or even millions of times faster than a poor one for the same problem.

Theorem 3.1 (Efficiency Matters)**.** *The choice of algorithm and data structure can affect program performance by* orders of magnitude*. A problem that takes hours with a poor algorithm might take milliseconds with a better one.*

Concrete Example*: Searching in 1,000,000 elements*

- *Linear search: up to 1,000,000 comparisons*
- *Binary search (on sorted data): at most 20 comparisons*
- *Hash table: typically 1 comparison*

The ratio $1,000,000 : 20 : 1$ *demonstrates why algorithm choice is critical.*

Important

Real-world systems often handle millions or billions of data items. The difference between $O(n)$ and $O(\log n)$ might be irrelevant for 10 items, but it's the difference between instant response and multi-hour processing for large-scale data.

Exercise

Exercise 1.4: Consider a food delivery mobile app that needs to:

1. Display restaurants sorted by distance from user
2. Show real-time delivery driver locations on a map
3. Calculate optimal delivery routes
4. Handle thousands of concurrent users

For each requirement, identify:

- What algorithmic challenges exist?
- What data structures might be needed?
- What trade-offs (speed vs. memory, accuracy vs. simplicity) must be considered?

4 Algorithm Analysis: The Basics

Story

Two students, Andi and Budi, both write programs to sort a list of numbers. When they test with 10 numbers, both programs run instantly. "Great!" they think. "We're done!"
But when their professor tests their code with 10,000 numbers, Andi's program finishes in a second while Budi's takes 2 hours. What happened?
Budi's program worked correctly—but it didn't *scale*. This is why we analyze algorithms: to predict how they'll perform as the problem size grows. We need mathematical tools to evaluate efficiency without running experiments every time.

4.1 Why Analyze Algorithms?

We analyze algorithms to:

1. **Predict performance** before implementation
2. **Compare** different approaches objectively
3. **Identify bottlenecks** in existing code
4. **Ensure scalability** as data grows

Key Point

Algorithm analysis focuses on how running time grows as input size increases, not the exact time in seconds (which depends on hardware, programming language, etc.). We care about the *rate of growth*.

4.2 Measuring Efficiency

Definition 4.1 (Time Complexity)**.** *The **time complexity** of an algorithm is a function $T(n)$ that describes the number of basic operations executed by the algorithm as a function of input size n.*

We focus on the asymptotic behavior *as n grows large, using **Big-O notation**.*

4.3 The Word-RAM Computation Model

To analyze algorithms rigorously, we need a model of computation that defines which operations are "basic" and cost $O(1)$ time.

Definition 4.2 (Word-RAM Model)**.** *The **Word-RAM model** assumes:*

- *Memory is an array of **words**, each consisting of w bits.*
- *Each word can store one integer, one pointer/address, or one value.*
- *Operations on a single word take **constant time** ($O(1)$), including:*
  - *Arithmetic: $+$, $-$, $\times$, integer division, comparisons*
  - *Memory access: reading or writing one memory cell by address*
  - *Bitwise operations: AND, OR, XOR, shifts*
- *The word size is $w = \Theta(\log n)$, where n is the input size.*

Key Point

Why $w = \Theta(\log n)$? If the input has n elements, we need to be able to address at least n distinct memory locations. To represent n different addresses, we need $2^w \geq n$, which gives $w \geq \log_2 n$. So the word size grows logarithmically with input size.

Note

Visual intuition of the Word-RAM model:

```
              +-----------+
              |    CPU    |
              +-----------+
                     |
                     |  w-bit address
                     v
            +-------------------+
Address 0 ->|   Word 0 (w bits) |
Address 1 ->|   Word 1 (w bits) |
Address 2 ->|   Word 2 (w bits) |
    ...     |        ...        |
Address n-1>| Word n-1 (w bits) |
            +-------------------+
```

Key idea: Each memory address fits in one word, so accessing any element by index costs $O(1)$.

Example 4.1 (Counting Operations in Word-RAM)**.** *Consider summing an array of n integers:*

```
sum = 0
for i from 1 to n:
    sum = sum + A[i]
```

In the Word-RAM model, each loop iteration performs constant-time word operations (one memory read, one addition, one write), so each iteration costs $O(1)$. Therefore:

$$T(n) = c_1 + c_2 \cdot n = O(n)$$

The key insight: we count word-level primitive operations*, not exact CPU cycles.*

4.4 Asymptotic Notation

Definition 4.3 (Big-O Notation)**.** *We say $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that:*

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

Informally, $f(n) = O(g(n))$ means $f(n)$ grows no faster than $g(n)$ as n becomes large.

Note

Think of Big-O as describing the "shape" of how running time grows:

- $O(1)$: Constant (flat line—doesn't grow)
- $O(\log n)$: Logarithmic (grows very slowly)
- $O(n)$: Linear (grows proportionally)
- $O(n \log n)$: Linearithmic (slightly worse than linear)
- $O(n^2)$: Quadratic (grows rapidly)
- $O(2^n)$: Exponential (grows extremely rapidly)

Important

When using Big-O notation:

- Drop constant factors: $2n$ becomes $O(n)$, not $O(2n)$
- Drop lower-order terms: $n^2 + n$ becomes $O(n^2)$, not $O(n^2 + n)$
- Focus on worst-case behavior (unless stated otherwise)

We do this because Big-O describes growth *trends*, not exact values. For large n, the highest-order term dominates.

Example 4.2 (Analyzing FindMax Algorithm)**.** *Recall our FindMax algorithm:*

1. *Let $max \leftarrow A[1]$* *[1 operation]*

2. *For $i \leftarrow 2$ to n:* *[$n-1$ iterations]*
   (a) *If $A[i] > max$, then $max \leftarrow A[i]$* *[1-2 operations per iteration]*
3. *Return max* *[1 operation]*

***Analysis**:*

- *Step 1: 1 operation*
- *Step 2: $(n-1)$ iterations, each with 1 comparison and possibly 1 assignment*
- *Step 3: 1 operation*

Total operations: $T(n) = 1 + 2(n-1) + 1 = 2n$
Therefore: $T(n) = O(n)$ (linear time complexity)
***Interpretation**: Doubling the input size doubles the running time.*

Exercise

Exercise 1.5: Analyze the time complexity of the following algorithm that checks if an array contains duplicate elements:

```
Algorithm: HasDuplicates(A, n)
for i from 1 to n:
    for j from i+1 to n:
        if A[i] == A[j]:
            return TRUE
return FALSE
```

Questions to answer:

(a) How many times does the outer loop execute?

(b) For each iteration of the outer loop, how many times does the inner loop execute?

(c) What is the total number of comparisons in the worst case?

(d) What is the time complexity in Big-O notation?

(e) How could you improve the algorithm to make it faster?

4.5 Common Growth Rates

Here's a comparison of common time complexities with concrete numbers:

Important

Notice how dramatically exponential algorithms ($O(2^n)$) become infeasible. Even with the fastest computer, an $O(2^n)$ algorithm is impractical for $n > 30$ or so. This is why we seek polynomial-time algorithms whenever possible.

Complexity	**Name**	$n = 100$	$n = 1000$	$n = 10^6$
$O(1)$	Constant	1	1	1
$O(\log n)$	Logarithmic	7	10	20
$O(n)$	Linear	100	1,000	10^6
$O(n \log n)$	Linearithmic	700	10,000	2×10^7
$O(n^2)$	Quadratic	10,000	10^6	10^{12}
$O(2^n)$	Exponential	10^{30}	Infeasible	Infeasible

Table 1: Growth rates of common time complexities

4.6 Big-Omega and Big-Theta Notation

Big-O gives an *upper bound* on growth, but sometimes we also need a *lower bound* or a *tight bound*.

Definition 4.4 (Big-Omega Notation (Ω) — Lower Bound)**.** *We say $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:*

$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0$$

Informally, $f(n) = \Omega(g(n))$ means $f(n)$ grows at least as fast *as $g(n)$.*

Definition 4.5 (Big-Theta Notation (Θ) — Tight Bound)**.** *We say $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$* ***and*** *$f(n) = \Omega(g(n))$.*

Equivalently, there exist positive constants c_1, c_2, and n_0 such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0$$

Informally, $f(n) = \Theta(g(n))$ means $f(n)$ grows at exactly the same rate *as $g(n)$.*

Key Point

Comparing the three notations:

Notation	**Meaning**	**Analogy**
$O(g(n))$	Upper bound (at most)	$\leq$
$\Omega(g(n))$	Lower bound (at least)	$\geq$
$\Theta(g(n))$	Tight bound (exactly)	$=$

Big-Theta gives the *most precise* characterization of an algorithm's growth rate.

Example 4.3 (Using All Three Notations)**.** *Consider $f(n) = 3n^2 + 5n + 2$:*

- $f(n) = O(n^2)$ ✓ *(upper bound: $3n^2 + 5n + 2 \leq 4n^2$ for large n)*
- $f(n) = O(n^3)$ ✓ *(also a valid upper bound, but not tight)*
- $f(n) = \Omega(n^2)$ ✓ *(lower bound: $3n^2 + 5n + 2 \geq 3n^2$ for all n)*
- $f(n) = \Omega(n)$ ✓ *(also a valid lower bound, but not tight)*
- $f(n) = \Theta(n^2)$ ✓ *(tight bound — this is the most informative)*

Exercise

Exercise 1.7: Determine the tightest Big-Theta (Θ) complexity for each:

(a) $f(n) = 5n + 100$

(b) $f(n) = n^2 + 1000n + 50$

(c) $f(n) = 2^n + n^3$

(d) $f(n) = \log n + \sqrt{n}$

Hint: The highest-order term dominates for large n.

4.7 Best, Average, and Worst Case

For many algorithms, the running time depends not only on the *size* of the input but also on the *specific* input.

Definition 4.6 (Case Analysis).

- ***Best case***: *The minimum number of operations over all inputs of size n.*
- ***Worst case***: *The maximum number of operations over all inputs of size n.*
- ***Average case***: *The expected number of operations over a "typical" or random input of size n.*

Example 4.4 (Case Analysis of Linear Search). *Searching for a target in an unsorted array of n elements:*

- ***Best case***: $O(1)$ — *target is the first element.*
- ***Worst case***: $O(n)$ — *target is the last element or not present.*
- ***Average case***: $O(n)$ — *on average, we check about $n/2$ elements.*

Key Point

We usually focus on **worst-case analysis** because:

- It provides a *guarantee*: the algorithm will never be slower than the worst case.
- Average-case analysis requires assumptions about input distribution, which may not hold in practice.
- It is usually easier to determine than average-case.

However, average-case analysis is sometimes more useful (e.g., QuickSort has $O(n^2)$ worst case but $O(n \log n)$ average case, which is why it's widely used in practice).

Exercise

Exercise 1.8: Consider the FindMax algorithm from Section 1.

(a) What is the best-case time complexity? Does the input arrangement affect it?

(b) What is the worst-case time complexity?

(c) Is there a difference between best and worst case for this algorithm? Why or why not?

Hint: Think about how many comparisons are always performed, regardless of the input values.

5 Problem-Solving Strategies

Story

The ancient Greeks had a puzzle: Given a list of cities and the distances between them, can you visit every city exactly once and return home, minimizing total distance?
This "Traveling Salesman Problem" stumped mathematicians for centuries. Why? Because the naive approach—trying every possible route—grows exponentially. For just 20 cities, there are $20! \approx 2.4 \times 10^{18}$ possible routes!
This is where problem-solving strategies become crucial. When brute force fails, we need clever approaches: divide and conquer, greedy methods, dynamic programming. Understanding these strategies is like having a Swiss Army knife for tackling hard problems.

5.1 Common Strategies

1. **Brute Force**
   - Try all possible solutions
   - Simple to understand and implement
   - Often too slow for large inputs
   - Useful for: Small problems, verification, baseline comparison
2. **Divide and Conquer**
   - Break problem into smaller subproblems
   - Solve subproblems recursively
   - Combine solutions to solve original problem
   - Examples: Binary search, merge sort, quicksort
3. **Greedy Method**

- Make locally optimal choice at each step
- Hope local choices lead to global optimum
- Fast but doesn't always give best solution
- Examples: Dijkstra's algorithm, Huffman coding

4. **Dynamic Programming**
   - Solve each subproblem once and store result
   - Reuse stored solutions when needed
   - Trade space for time
   - Examples: Fibonacci numbers, shortest paths

Key Point

No single strategy works for all problems. Part of becoming a skilled programmer is learning to recognize which strategy (or combination of strategies) fits a particular problem.

Example 5.1 (Divide and Conquer: Binary Search)**.** ***Problem****: Find a target value in a sorted array.*

Naive Approach *(Linear Search): Check each element until found or end reached. Time: $O(n)$.*

Divide and Conquer Approach *(Binary Search):*

1. *Check middle element*
2. *If target equals middle, done!*
3. *If target < middle, search left half*
4. *If target > middle, search right half*
5. *Repeat until found or no elements remain*

Algorithm*:* BinarySearch(A, *target*, *left*, *right*)

```
if left > right:
    return NOT_FOUND
mid = (left + right) / 2
if A[mid] == target:
    return mid
else if target < A[mid]:
    return BinarySearch(A, target, left, mid-1)
else:
    return BinarySearch(A, target, mid+1, right)
```

Analysis*:*

- *Each comparison eliminates half the remaining elements*
- *After k comparisons, $\frac{n}{2^k}$ elements remain*

- *Worst case:* $k = \log_2 n$ *comparisons*
- *Time complexity:* $O(\log n)$

***Why It Works**: The sorted property allows us to eliminate half the search space each time—a huge efficiency gain!*

Note

Let's trace binary search on array $A = [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78]$ searching for target = 23:

1. Range: [0, 10], mid = 5, $A[5] = 23$. Found! (1 comparison)

Now let's search for target = 45:

1. Range: [0, 10], mid = 5, $A[5] = 23 < 45 \rightarrow$ search right
2. Range: [6, 10], mid = 8, $A[8] = 56 > 45 \rightarrow$ search left
3. Range: [6, 7], mid = 6, $A[6] = 38 < 45 \rightarrow$ search right
4. Range: [7, 7], mid = 7, $A[7] = 45$. Found! (4 comparisons)

Compare this to linear search, which would need 8 comparisons to find 45!

Exercise

Exercise 1.6: For each problem below, suggest an appropriate problem-solving strategy and explain why:

(a) Finding the shortest path in a maze

(b) Scheduling tasks with deadlines to maximize the number of completed tasks

(c) Computing the n-th Fibonacci number where $F(n) = F(n-1) + F(n-2)$

(d) Sorting a deck of cards

(e) Finding the longest common subsequence between two DNA sequences

Hint: Consider whether the problem has:

- Natural recursive structure (divide and conquer)
- Overlapping subproblems (dynamic programming)
- Greedy choice property (greedy method)
- Small enough size for exhaustive search (brute force)

6 Summary Tables and Quick Reference

6.1 Algorithm Properties Checklist

Property	**Verification Question**
Finiteness	Does the algorithm terminate after finite steps?
Definiteness	Is each step precisely and unambiguously defined?
Input	What are the inputs? How many?
Output	What outputs are produced?
Effectiveness	Can all operations be performed exactly in finite time?

Table 2: Checklist for Verifying Algorithm Properties

6.2 Common Time Complexities

Complexity	**Name**	**Example**	$n = 10^6$
$O(1)$	Constant	Array access	1 operation
$O(\log n)$	Logarithmic	Binary search	≈ 20 operations
$O(n)$	Linear	Linear search	10^6 operations
$O(n \log n)$	Linearithmic	Merge sort	$\approx 2 \times 10^7$ operations
$O(n^2)$	Quadratic	Bubble sort	10^{12} operations
$O(2^n)$	Exponential	Recursive tree	Infeasible

Table 3: Common Time Complexities with Examples

6.3 Data Structure Operation Costs

Structure	**Access**	**Search**	**Insert**	**Delete**
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Sorted Array	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash Table	N/A	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
Binary Search Tree	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg

Table 4: Time Complexity of Common Data Structure Operations

7 Practice Problems

7.1 Basic Concepts

Exercise

Problem 1.1: Write a precise algorithm (in pseudocode) to determine if a given year is a leap year. Recall the rules:

- A year is a leap year if it's divisible by 4
- EXCEPT years divisible by 100 are not leap years
- UNLESS the year is also divisible by 400

Verify your algorithm satisfies all five algorithm properties.

Exercise

Problem 1.2: An algorithm has time complexity $T(n) = 3n^3 + 5n^2 + 100n + 50$.

(a) Express this in Big-O notation

(b) Justify why lower-order terms can be dropped

(c) If this algorithm takes 1 second for $n = 100$, approximately how long will it take for $n = 200$?

7.2 Algorithm Analysis

Exercise

Problem 1.3: Consider the following algorithm:

```
Algorithm: Mystery(A, n)
for i from 1 to n:
    for j from 1 to i:
        sum = sum + A[i] * A[j]
return sum
```

(a) What does this algorithm compute?

(b) How many multiplications are performed?

(c) What is the time complexity in Big-O notation?

(d) Trace through the algorithm with $A = [2, 3, 5]$

Exercise

Problem 1.4: You have three algorithms for the same problem:

- Algorithm A: $T(n) = 100n$
- Algorithm B: $T(n) = n^2$
- Algorithm C: $T(n) = 10n \log n$

(a) Which is fastest for $n = 10$?

(b) Which is fastest for $n = 1000$?

(c) For what value of n (approximately) does Algorithm C become faster than Algorithm A?

(d) Which algorithm would you choose for a production system that might handle any input size? Why?

7.3 Data Structures

Exercise

Problem 1.5: Design a data structure to implement an "undo" feature in a text editor. Your structure should support:

- `recordChange(change)`: Save a change
- `undo()`: Undo the most recent change
- `canUndo()`: Check if undo is possible

(a) What data structure would you use? Why?

(b) Write pseudocode for each operation

(c) What is the time complexity of each operation?

(d) How would you extend this to support "redo"?

7.4 Problem-Solving Strategies

Exercise

Problem 1.6: The following problem can be solved using divide and conquer:
Given a sorted array where every element appears exactly twice except for one element that appears once, find that unique element.
Example: $A = [1, 1, 3, 3, 4, 5, 5, 7, 7, 8, 8] \rightarrow$ Answer: 4

(a) What would be the time complexity of a naive solution?

(b) Describe a divide and conquer approach

(c) Write pseudocode for your approach

(d) What is the time complexity? Space complexity?

7.5 Challenge Problem

Exercise

Challenge Problem 1.7:
You're given an array of n integers where each integer is in the range $[1, n]$. The array contains $n + 1$ elements, so by the Pigeonhole Principle, at least one number must be duplicated.
Find a duplicate number in $O(n)$ time using only $O(1)$ extra space. You may not modify the array.
Example: $A = [1, 3, 4, 2, 2] \rightarrow$ Answer: 2
Hints:

- Think of the array as defining a function $f(i) = A[i]$
- This creates a sequence: $f(0), f(f(0)), f(f(f(0))), \ldots$
- The duplicate creates a cycle in this sequence
- Research: Floyd's Cycle Detection Algorithm ("tortoise and hare")

This is a difficult problem that combines several algorithmic concepts. Don't be discouraged if it takes time to solve!

8 Closing Thoughts

Story

At the end of this module, I want you to remember this: Algorithms and data structures aren't just academic concepts—they're the fundamental tools that make modern computing possible.

Every time you unlock your phone with facial recognition, stream a video, or get driving directions, you're benefiting from decades of algorithmic innovation. The sorting algorithms we'll study were developed in the 1960s and still power systems today. The data structures we'll explore enable everything from databases to operating systems.

But here's the most important part: *you* are now equipped to think algorithmically. When faced with a problem, you won't just write code that works—you'll write code that works *efficiently*. You'll ask: "Is there a better way?" And increasingly, you'll know how to find that better way.

Welcome to the journey of algorithmic thinking. It will transform not just how you program, but how you solve problems in any domain.

"An algorithm must be seen to be believed."
— Donald Knuth